

Backgammon

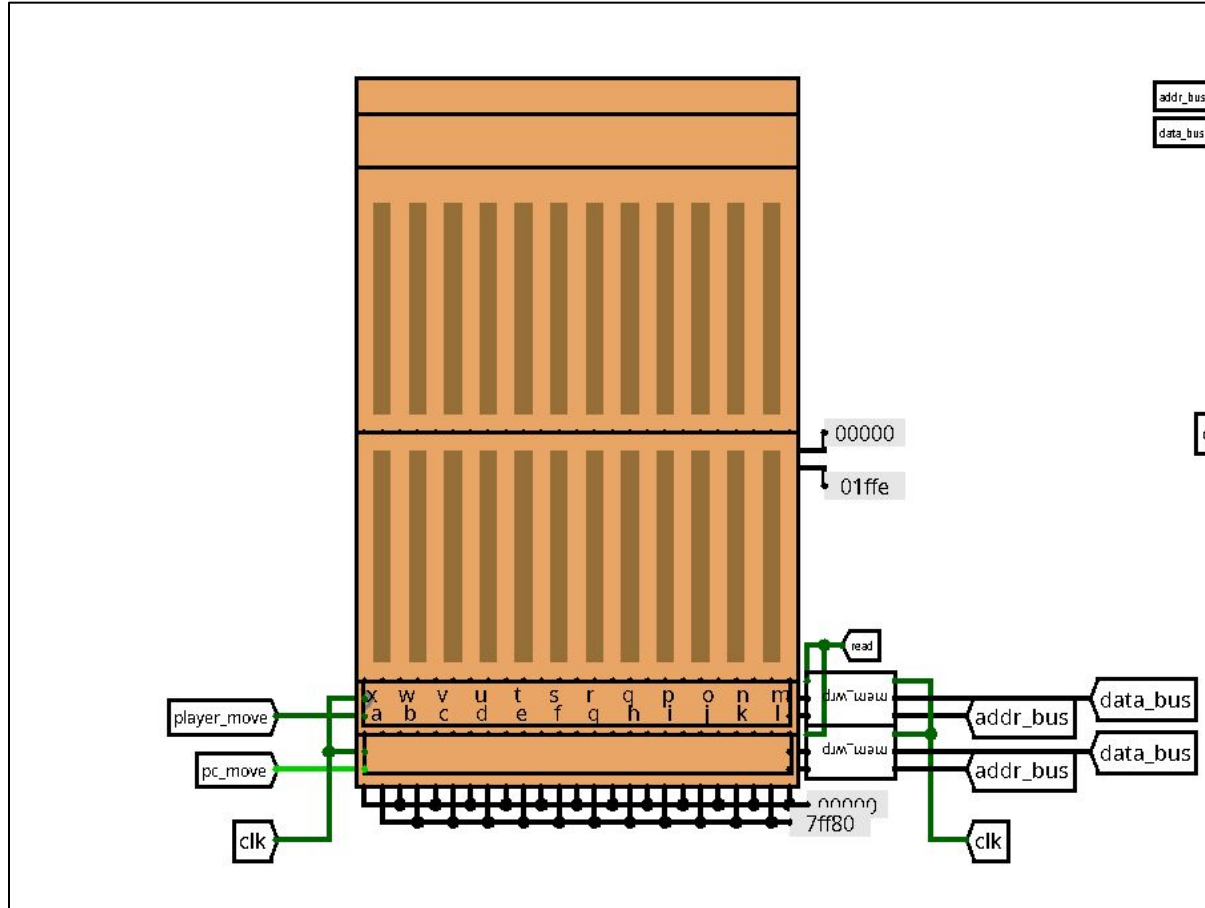
Group 25214
Dedov Dmitry
Starikova Maria
Medvedev Mikhail

Game Rules

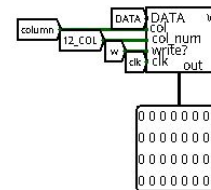
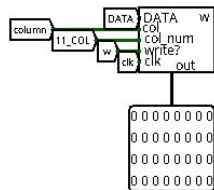
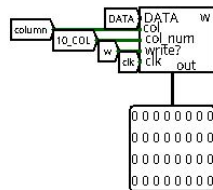
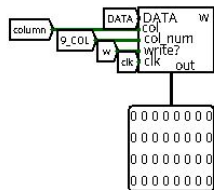
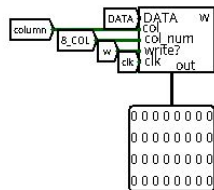
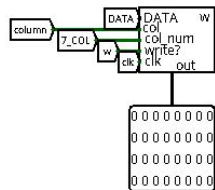
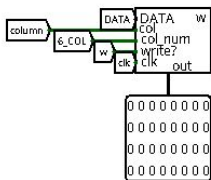
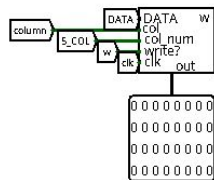
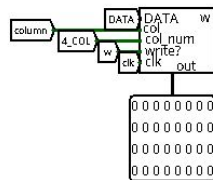
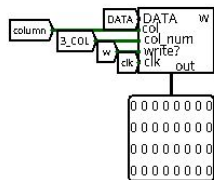
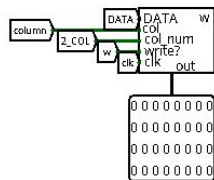
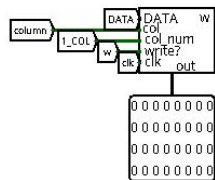
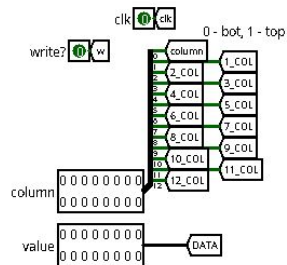
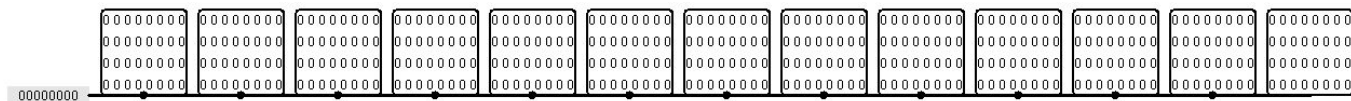
- Goal: Be the first to bear off all 11 checkers
- Start: All checkers are placed on “head” point; Human player makes first move
- Making a move: Roll 2 dice and move your checkers forward by those numbers; Doubles → 4 moves instead of 2; No legal move available → pass manually
- Restrictions: Only 1 checker can be moved from start point per roll (2 if Doubles); Zabor rule - no 6-checkers block unless opponent has started bearing off; No hitting - cannot land a point, which already landed by opponent
- Bearing off: all your 11 checkers are in home; Exact die value, higher value allowed if point is empty and no checkers behind

Hardware

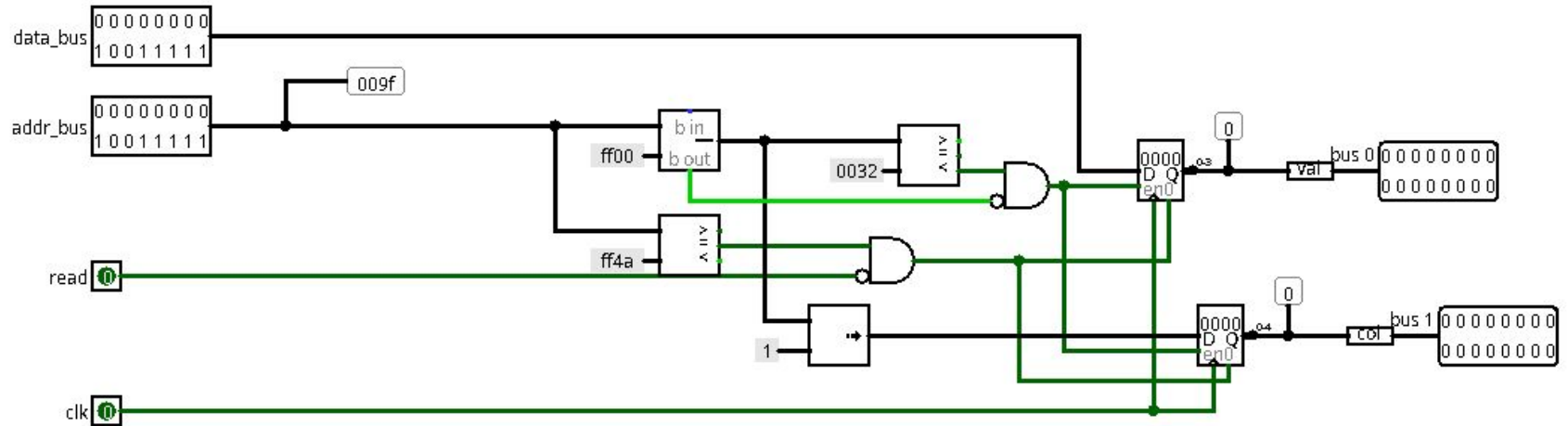
LED Matrix System



Video Controller

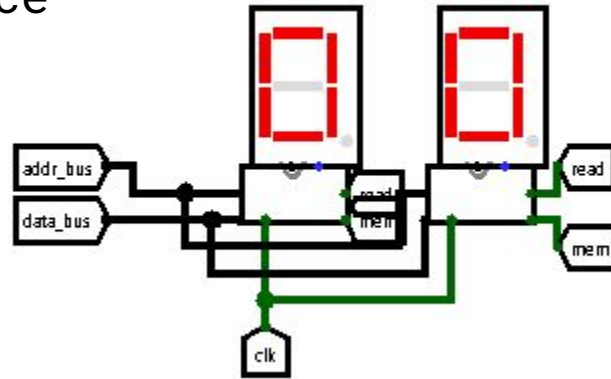


Memory Wrapper

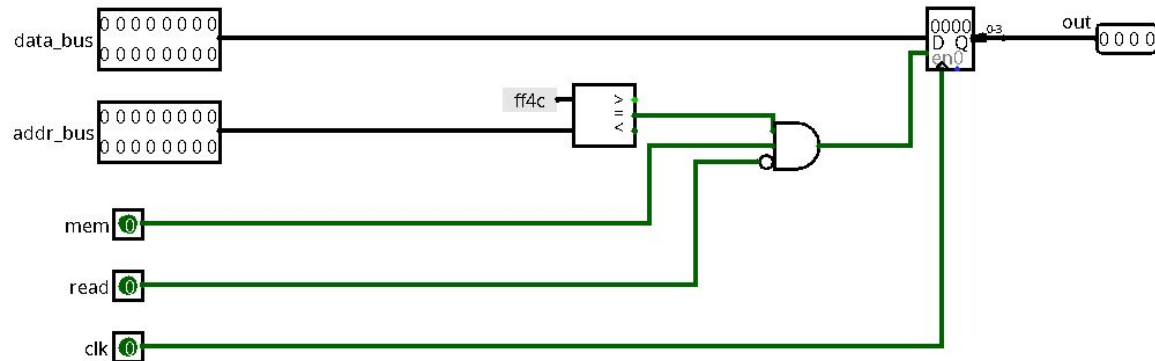


Dice

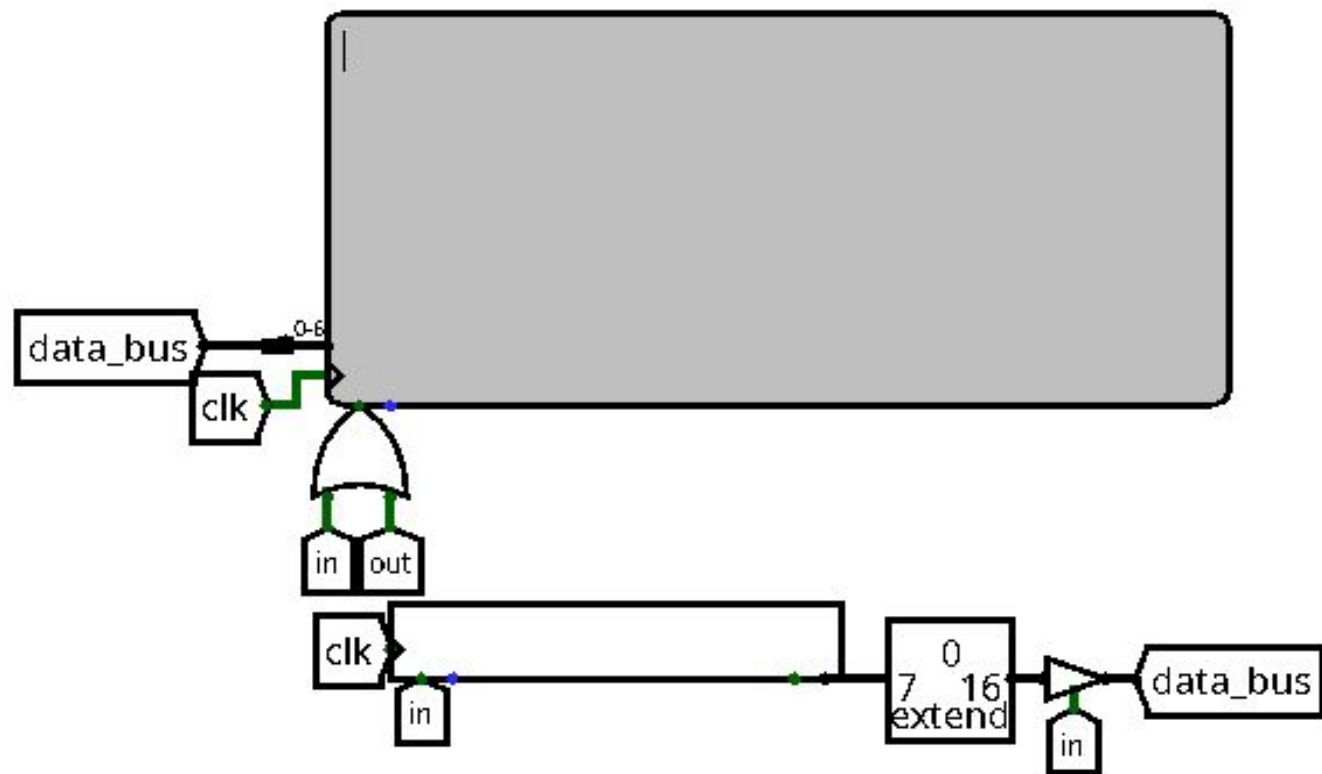
Dice



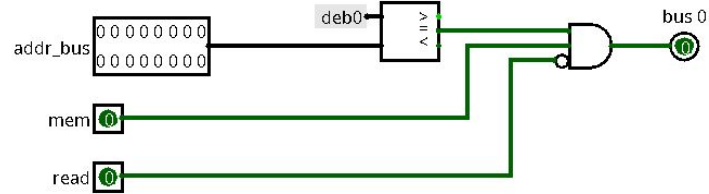
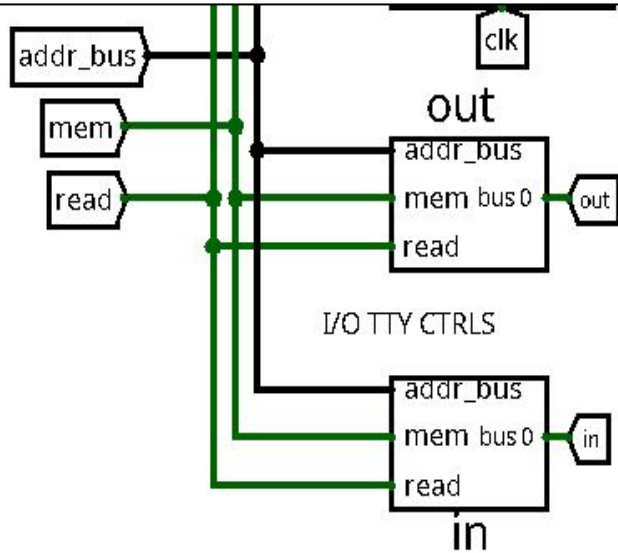
Dice display controller



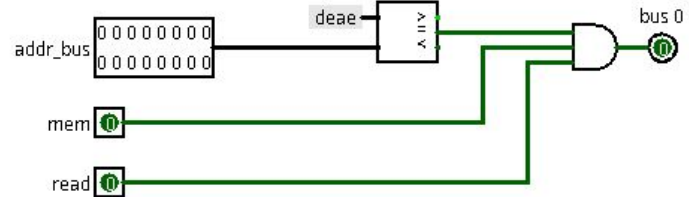
Input/Output



Input/Output controllers



Output controller



Input controller

Software

Game initialization

Our implementation is a bit simplified, each player has 11 checkers instead of 15

```
3  ~ int main(){
4
5      // init
6      _player = 1;
7      _amt_of_checkers[1] = 11;
8      _points[1] = 11;
9      _colors[0] = 1;
10     _player = 0;
11     _amt_of_checkers[0] = 11;
12     _points[13] = 11;
13     _colors[12] = 2;
14     _player = -1;
15     unsigned char rmv_chck = 0;
16     unsigned char ai_rmv_chck = 0;
17
18     // game loop
19     while(1){
20         player_move(&rmv_chck);
21         if (!_amt_of_checkers[1]) {
22             print_to_tty("\nPlayer win!");
23             break;
24         }
25         computer_move(&ai_rmv_chck);
26         if (!_amt_of_checkers[0]) {
27             print_to_tty("\nComputer win!");
28             break;
29         }
30     }
31
32 }
33
```

Player move

```
void player_move(unsigned char* can_remove_checker){
    unsigned char move[2];
    print_to_tty("\n--- Player Turn ---\n");
    randomize();
    _player = 1;

    if (!(*can_remove_checker)){
        if (is_all_in_home()){
            print_to_tty ("\nDBG: all checkers in home\nnow you bear off them");
            *can_remove_checker = 1;
        }
    }

    // _random - physical array, which you can see on displays
    // dice - actual array, which help us with doubles

    unsigned char dice[4];
    int dice_count = 0;
    int head_can_taken = 0;

    char d1 = _random[0];
    char d2 = _random[1];

    if (d1 == d2) {
        print_to_tty("\nRolling doubles!!");
        dice[0] = d1; dice[1] = d1; dice[2] = d1; dice[3] = d1;
        dice_count = 4;
        head_can_taken = 2;
    } else {
        dice[0] = d1; dice[1] = d2;
        dice_count = 2;
        head_can_taken = 1;
    }
}
```

Random

```
static unsigned int state = 44257;
```

```
unsigned int xorshift16() {  
    unsigned int x = state;  
    x ^= x << 7;  
    x ^= x >> 9;  
    x ^= x << 8;  
    state = x;  
    return x;  
}
```

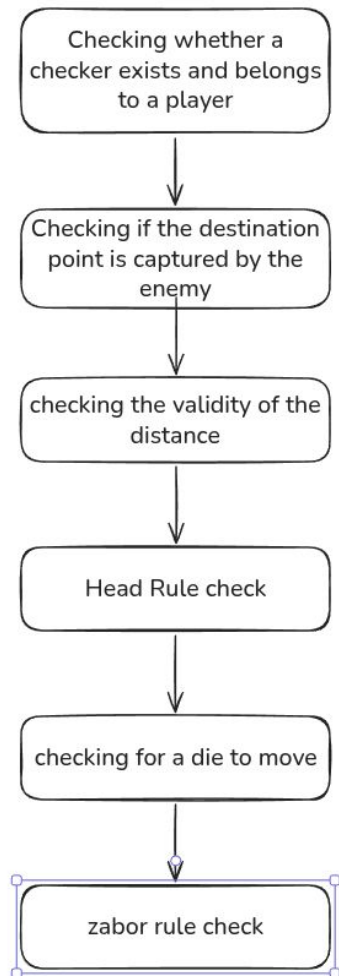
```
void randomize(){  
    for(int i = 0; i < 2; i++){  
        unsigned int val;  
        do {  
            val = xorshift16() & 0x07;  
        } while (val < 1 || val > 6);  
        _random[i] = (char)val;  
    }  
}
```

Player move

```
68 while (dice_count){
69     print_to_tty("\nMoves left: ");
70     *(char*)TTY = dice_count + '0';
71
72     // * fetch coordinates
73     print_to_tty("\nfrom (z to pass): ");
74     move[0] = getc();
75     if (move[0] == 'z' - 'a') {
76         print_to_tty("\nPassed.\n");
77         break;
78     }
79
80     if (*can_remove_checker) print_to_tty("\nto: (y to bear off) ");
81     else print_to_tty("\nto: ");
82     move[1] = getc();
83
84     if (move[1] == 24){
85         if (!(*can_remove_checker)){
86             print_to_tty("You cannot bear off now");
87             continue;
88         } else{
89             int used_dice = validate_bear_off(move[0], dice, dice_count);
90             if(!used_dice) continue;
91             remove_checker(move[0]);
92             for(int i = 0; i < dice_count; i++){
93                 if (dice[i] == used_dice) {
94                     dice[i] = dice[--dice_count];
95                     break;
96                 }
97             }
98         }
99     }
100     if (d1 != d2) {
101         if (_random[0] == used_dice) _random[0] = 0;
102         else if (_random[1] == used_dice) _random[1] = 0;
103     } else if (dice_count == 2) _random[1] = 0;
104 }
105 } else {
```

Player move

```
32 // normal move, when can_remove_checker down
31 print_to_tty("\nMove validation...");
30
29 if (is_move_valid(move, dice, dice_count, head_can_taken)){
28     move_checker(move);
27     if (!zabor_rule()){
26         unsigned char undomove[2] = {move[1], move[0]};
25         move_checker(undomove);
24     } else {
23         print_to_tty("\nOk...");
22         if (move[0] == 0) head_can_taken--;
21
20         int dist = get_dst(move[0], move[1], _player);
19         for(int i = 0; i < dice_count; i++){
18             if (dice[i] == dist) {
17                 dice[i] = dice[dice_count - 1];
16                 dice_count--;
15                 break;
14             }
13         }
12
11         if (d1 != d2) {
10             if (_random[0] == dist) _random[0] = 0;
9             else if (_random[1] == dist) _random[1] = 0;
8             } else if (dice_count == 2) _random[1] = 0;
7         }
6     } else {
5         print_to_tty("\nInvalid\n");
4     }
3 }
2 }
1 _player = -1;
```



```
char is_move_valid(unsigned char* move, unsigned char* dice, int dice_count, int head_can_taken){
    int color = (_player == 1) ? 1 : 2;
    if (_colors[move[0]] != color || !_points[move[0]+1]){
        if (color == 1) print_to_tty("\nErr: Not your checker!\n");
        return 0;
    }

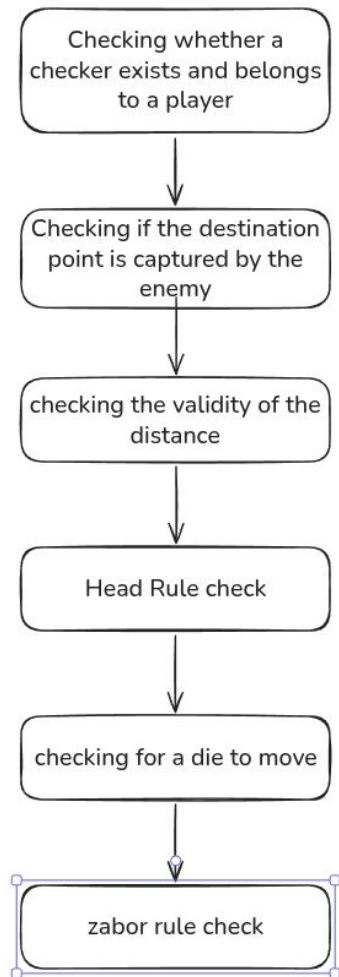
    char opponent = (_player == 1) ? 2 : 1;
    if (_colors[move[1]] == opponent) {
        if (color == 1) print_to_tty("\nErr: Point captured by opponent!\n");
        return 0;
    }

    int dist = get_dst(move[0], move[1], _player);
    if (dist <= 0 || dist > 6) {
        if (color == 1) print_to_tty("\nErr: Invalid dist\n");
        return 0;
    }

    char head_pos = (_player == 1) ? 0 : 12;
    if (move[0] == head_pos && head_can_taken == 0) {
        if (color == 1) print_to_tty("\nErr: Head rule\n");
        return 0;
    }

    char can_move = 0;
    for(int i = 0; i < dice_count; i++){
        if (dice[i] == dist) {
            can_move = 1;
            break;
        }
    }
    if (!can_move) {
        if (color == 1) print_to_tty("\nErr: Dice doesn't exists\n");
        return 0;
    }

    return 1;
}
```

```

10 char zabor_rule() {
11     if (_amt_of_checkers[!_player] < 11) return 1;
12
13     char opponent = (_player == 1) ? 2 : 1;
14     char color = (_player == 1) ? 1 : 2;
15     char opp_finish = (opponent == 2) ? 11 : 23;
16     for (int i = 0; i < 24; i++) {
17         int count = 0;
18         for (int j = 0; j < 6; j++) {
19             int idx = i + j;
20             if (idx >= 24) idx -= 24;
21
22             if (_colors[idx] == color)
23                 count++;
24             else
25                 break;
26         }
27         if (count == 6) {
28             char dst = get_dst(i+6, opp_finish, 0);
29             char found = 0;
30             // trying to find opponent chip ahead
31             for (int j = 0; j <= dst; j++){
32                 int curr = i+6+j;
33                 if (curr >= 24) curr -= 24;
34                 if (curr >= 24) curr -= 24;
35                 if (_colors[curr] == opponent){
36                     found = 1;
37                     break;
38                 }
39             }
40
41             if (!found) {
42                 print_to_tty("\nErr: Illegal block (6 in row)\n");
43                 return 0;
44             }
45         }
46         i += count;
47     }
48
49     return 1;
50 }
51

```

Checking whether a checker exists and belongs to a player



checking for a die to bear off



if doesn't exist trying to apply the senior rule

```
197 char validate_bear_off(unsigned char from, unsigned char* dice, int dice_count) {
198     int color = (_player == 1) ? 1 : 2;
199     if (_colors[from] != color || _points[from + 1] == 0) {
200         print_to_tty("\nErr: Not your checker!\n");
201         return 0;
202     }
203 }
204
205 // get dst
206 int dst = (_player == 1) ? (24 - from) : (12 - from);
207
208 // find dice
209 for (int i = 0; i < dice_count; i++) {
210     if (dice[i] == dst) {
211         return dst; // return dice
212     }
213 }
214
215 // check checkers behind
216 char checkers_behind = 0;
217 int home_start = (_player == 1) ? 18 : 6;
218 int player = _player;
219 _player = -1;
220 for (int i = home_start; i < from; i++) {
221     if (_colors[i] == player && _points[i + 1] > 0) {
222         checkers_behind = 1;
223         break;
224     }
225 }
226 _player = player;
227 // senior dice rule
228 if (!checkers_behind) {
229     int best_dice = 7;
230     for (int i = 0; i < dice_count; i++) {
231         if (dice[i] > dst && dice[i] < best_dice) {
232             best_dice = dice[i];
233         }
234     }
235     if (best_dice != 7) {
236         return best_dice;
237     }
238 }
239
240 print_to_tty("\nErr: No valid dice to bear off\n");
241 return 0;
242 }
```

Computer move

```
void computer_move(unsigned char* can_remove_checker) {
    unsigned char move[2];
    print_to_tty("\n--- Computer Turn ---\n");
    randomize();
    _player = 0;

    if (!(*can_remove_checker)){
        if (is_all_in_home()){
            print_to_tty ("\nDBG: Computer now can bearing off!");
            *can_remove_checker = 1;
        }
    }

    // _random - physical array, which you can see on displays
    // dice - actual array, which help us with doubles

    unsigned char dice[4];
    int dice_count = 0;
    int head_can_taken = 0;

    char d1 = _random[0];
    char d2 = _random[1];

    // * process double
    if (d1 == d2) {
        print_to_tty("\nRolling doubles!!");
        dice[0] = d1; dice[1] = d1; dice[2] = d1; dice[3] = d1;
        dice_count = 4;
        head_can_taken = 2;
    } else {
        dice[0] = d1; dice[1] = d2;
        dice_count = 2;
        head_can_taken = 1;
    }
}
```

Computer move

```
26
27 while (dice_count){
28     int bst_score = -1;
29     char bst_from = -1;
30     char bst_to = -1;
31
32     char best_is_bear_off = 0;
33     int dice_for_bear_off = 0;
34
35     for (int i = 0; i < dice_count; i++) {
36         int curr = dice[i];
37
38         for (char from = 0; from < 24; from++) {
39             if (_colors[from] != 2) continue;
40
41             char to = from + curr;
42             char is_bear_off = 0;
43
44             if (from >= 12) {
45                 if (to >= 24) to -= 24;
46             } else {
47                 if (to >= 12) {
48                     if (*can_remove_checker){
49                         to = 24;
50                         is_bear_off = 1;
51                     } else continue;
52                 }
53             }
54
55             move[0] = from;
56             move[1] = to;
57
```

Computer move

```
    if (is_bear_off) {
        int used_dice = validate_bear_off(from, dice, dice_count);
        if (used_dice) {
            int score = evaluate_move(from, to, 1);
            if (score > bst_score) {
                bst_score = score; bst_from = from; bst_to = to;
                best_is_bear_off = 1;
                dice_for_bear_off = used_dice;
            }
        }
    } else {
        if (is_move_valid(move, dice, dice_count, head_can_taken)) {
            int score = evaluate_move(from, to, 0);
            move_checker(move);
            if (zabor_rule()) {
                if (score > bst_score) {
                    bst_score = score; bst_from = from; bst_to = to;
                    best_is_bear_off = 0;
                }
            }
            unsigned char undomove[2] = {to, from};
            move_checker(undomove);
        }
    }
}

if (bst_score == -1) {
    print_to_tty("\nComputer passed.");
    break;
}

unsigned char best_move[2] = {bst_from, bst_to};
```

Move Evaluating

We tried to make the check 2 moves ahead, but even it is very slow in some situations.

```
static int evaluate_move(char from, char to, char is_bear_off) {  
    int score = 0;  
  
    if (is_bear_off) return 1000;  
  
    if (from == 12) score += 80;  
  
    if (_points[to + 1] == 0) {  
        score += 60;  
  
        if (to >= 0 && to <= 5) score += 40;  
    }  
    else {  
        if (_points[to + 1] >= 2) score -= 60;  
        else score += 10;  
    }  
  
    if (to > 0 && _colors[to - 1] == 2) score += 50;  
    if (to < 23 && _colors[to + 1] == 2) score += 50;  
  
    return score;  
}
```

Thanks for attention